



Indexing Text Documents and Information Extraction

IFN647 – Week 4

Professor Yuefeng Li
School of Computer Science
Queensland University of Technology

Learning Objectives

Topic ID	Key topics	Mapping to slides	Activities
1	Indexes and text representation	2-14	Q1
2	Index Construction	15-20	
3	Query Processing	21-27	Q2
4	Information extraction	28-30	
5	Hidden Markov Model	31-35	Q3, Q4
6	LLM-Based Information Extraction	36-39	Q5
	References	40	

1. Indexes and text representation

- Text representation aims to numerically represent the unstructured text documents to make them mathematically computable.
 - E.g., Bag of Words model (**BOW**) - uses all words appeared in the given document set as the index of the document vectors, e.g., the Boolean model, tf*idf, etc; or n-grams.
- Indexes* are data structures designed to make search faster.
- Text search has unique requirements, which leads to unique data structures.
- Most common data structure is ***inverted index***
 - general name for a class of structures
 - “inverted” because documents are associated with words, rather than words with documents.
 - An inverted index is the computational equivalent of the index found in the back of a textbook.

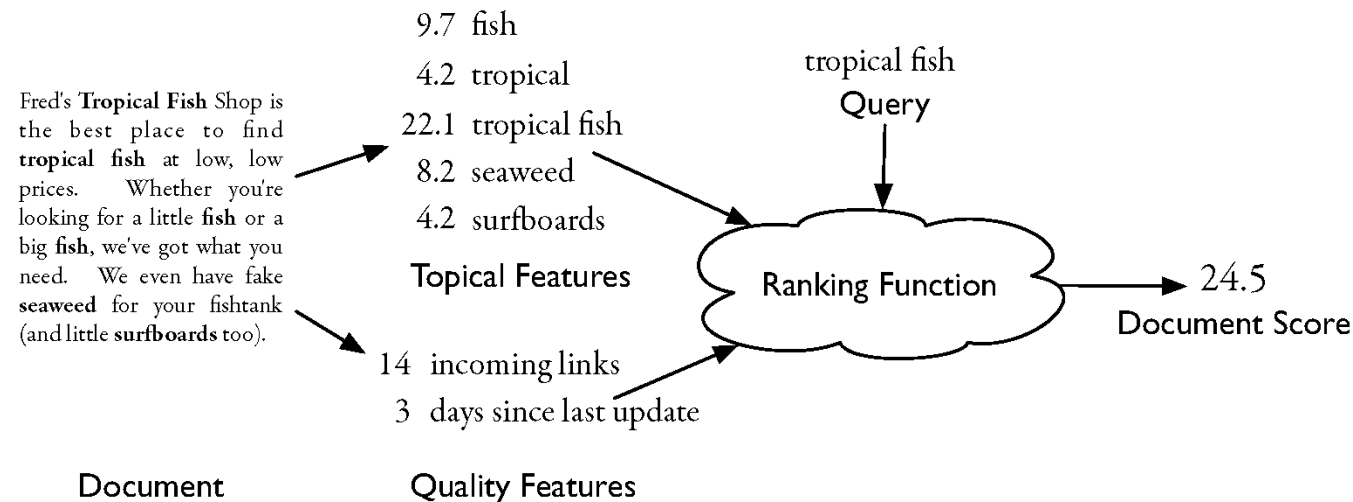
	$Term_1$	$Term_2$	$\dots$	$Term_t$
Doc_1	d_{11}	d_{12}	$\dots$	d_{1t}
Doc_2	d_{21}	d_{22}	$\dots$	d_{2t}
$\vdots$	$\vdots$			
Doc_n	d_{n1}	d_{n2}	$\dots$	d_{nt}

D_1 Tropical Freshwater Aquarium Fish.
 D_2 Tropical Fish, Aquarium Care, Tank Setup.
 D_3 Keeping Tropical Fish and Goldfish in Aquariums, and Fish Bowls.
 D_4 The Tropical Tank Homepage - Tropical Fish and Aquariums.

Terms	Documents			
	D_1	D_2	D_3	D_4
aquarium	1	1	1	1
bowl	0	0	1	0
care	0	1	0	0
fish	1	1	2	1
freshwater	1	0	0	0
goldfish	0	0	1	0
homepage	0	0	0	1
keep	0	0	1	0
setup	0	1	0	0
tank	0	1	0	1
tropical	1	1	1	2

Indexes and Ranking

- Indexes are designed to support *search or text analysis*
 - faster response time, supports updates
- Text search engines use a particular form of search: *ranking*
 - documents are retrieved in sorted order according to a score computing using the document representation, the query, and a *ranking algorithm*
- What is a reasonable abstract model for ranking?
 - enables discussion of indexes without details of retrieval model



Inverted Index

- Each index term is associated with an *inverted list*
 - Contains lists of documents, or lists of word occurrences in documents, and other information
 - Each entry is called a *posting*
 - The part of the posting that refers to a specific document or location is called a *pointer*
 - Each document in the collection is given a unique number
 - Lists are usually *document-ordered* (sorted by document number)

Example “Collection”

- S_1 Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.
- S_2 Fishkeepers often use the term tropical fish to refer only those requiring fresh water, with saltwater tropical fish referred to as marine fish.
- S_3 Tropical fish are popular aquarium fish, due to their often bright coloration.
- S_4 In freshwater fish, this coloration typically derives from iridescence, while salt water fish are generally pigmented.

Four sentences from the Wikipedia entry for *tropical fish*

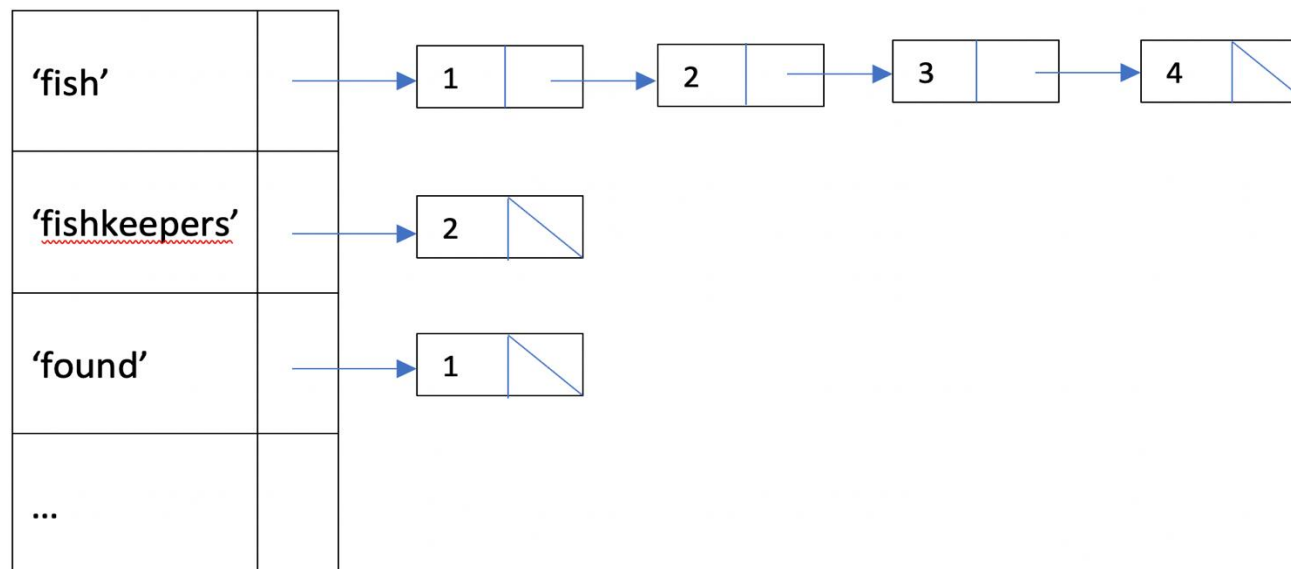
Simple Inverted Index

- This data structure records the documents in which each word appears.
- It does not record the number of times each word appears

and	1	only	2
aquarium	3	pigmented	4
are	3 4	popular	3
around	1	refer	2
as	2	referred	2
both	1	requiring	2
bright	3	salt	1 4
coloration	3 4	saltwater	2
derives	4	species	1
due	3	term	2
environments	1	the	1 2
fish	1 2 3 4	their	3
fishkeepers	2	this	4
found	1	those	2
fresh	2	to	2 3
freshwater	1 4	tropical	1 2 3
from	4	typically	4
generally	4	use	2
in	1 4	water	1 2 4
include	1	while	4
including	1	with	2
iridescence	4	world	1
marine	2		
often	2 3		

Data structures for the simple inverted index

- Linked List representation



- Dictionary representation (in Python)

```
I = {'fish': [1, 2, 3, 4], 'fishkeepers': [2], 'found': [1], ... }
```

Inverted Index with counts

- This data structure supports better ranking algorithms.
- We can calculate both term frequency and document frequency.

and	1:1					only	2:1
aquarium	3:1					pigmented	4:1
are	3:1	4:1				popular	3:1
around	1:1					refer	2:1
as	2:1					referred	2:1
both	1:1					requiring	2:1
bright	3:1					salt	1:1 4:1
coloration	3:1	4:1				saltwater	2:1
derives	4:1					species	1:1
due	3:1					term	2:1
environments	1:1					the	1:1 2:1
fish	1:2	2:3	3:2	4:2		their	3:1
fishkeepers	2:1					this	4:1
found	1:1					those	2:1
fresh	2:1					to	2:2 3:1
freshwater	1:1	4:1				tropical	1:2 2:2 3:1
from	4:1					typically	4:1
generally	4:1					use	2:1
in	1:1	4:1				water	1:1 2:1 4:1
include	1:1					while	4:1
including	1:1					with	2:1
iridescence	4:1					world	1:1
marine	2:1						
often	2:1	3:1					

Inverted Index with positions

- This data structure supports proximity matches.
- It also has the capability of previous data structures.

and	1,15					marine	2,22				
aquarium	3,5					often	2,2	3,10			
are	3,3	4,14				only	2,10				
around	1,9					pigmented	4,16				
as	2,21					popular	3,4				
both	1,13					refer	2,9				
bright	3,11					referred	2,19				
coloration	3,12	4,5				requiring	2,12				
derives	4,7					salt	1,16	4,11			
due	3,7					saltwater	2,16				
environments	1,8					species	1,18				
fish	1,2	1,4	2,7	2,18	2,23	term	2,5				
			3,2	3,6	4,3	the	1,10	2,4			
			4,13			their	3,9				
fishkeepers	2,1					this	4,4				
found	1,5					those	2,11				
fresh	2,13					to	2,8	2,20	3,8		
freshwater	1,14	4,2				tropical	1,1	1,7	2,6	2,17	3,1
from	4,8					typically	4,6				
generally	4,15					use	2,3				
in	1,6	4,1				water	1,17	2,14	4,12		
include	1,3					while	4,10				
including	1,12					with	2,15				
iridescence	4,9					world	1,11				

Proximity Matches

- Matching phrases or words within a window
 - e.g., "tropical fish", or "find tropical within 5 words of fish"
- Word positions in inverted lists make these types of query features efficient
 - e.g.,

tropical	1,1		1,7	2,6	2,17		3,1				
fish	1,2	1,4		2,7	2,18	2,23	3,2	3,6	4,3	4,13	

Fields and Extents

- Document structure is useful in search
 - *field* restrictions
 - e.g., date, from:, etc.
 - some fields more important
 - e.g., title
- Options:
 - separate inverted lists for each field type
 - add information about fields to postings
 - use *extent lists*

Extent Lists

- An *extent* is a contiguous region of a document
 - represent extents using word positions
 - inverted list records all extents for a given field type
 - e.g., (1,3) means S1 has a title that contains the first two words (ending just before 3rd word)

fish	1,2	1,4		2,7	2,18	2,23	3,2	3,6	4,3	4,13
title	1:(1,3)		2:(1,5)							4:(9,15)



extent list

Other Issues

- Pre-computed scores in inverted list
 - e.g., list for “fish” [(1:3.6), (3:2.2)], where 3.6 is total feature value for document 1
 - improves speed but reduces flexibility
- Score-ordered lists
 - query processing engine can focus only on the top part of each inverted list, where the highest-scoring documents are recorded
 - very efficient for single-word queries

Auxiliary Structures

- Inverted lists usually stored together in a single file for efficiency
 - *Inverted file*
- *Vocabulary or lexicon*
 - Contains a lookup table from index terms to the byte offset of the inverted list in the inverted file
 - Either hash table in memory or B-tree for larger vocabularies
- Term statistics stored at start of inverted lists.
- Collection statistics stored in a separate file.

2. Index Construction

- Simple in-memory indexer

```

procedure BUILDINDEX( $D$ )
   $I \leftarrow$  HashTable()
   $n \leftarrow 0$ 
  for all documents  $d \in D$  do
     $n \leftarrow n + 1$ 
     $T \leftarrow$  Parse( $d$ )
    Remove duplicates from  $T$ 
    for all tokens  $t \in T$  do
      if  $I_t \notin I$  then
         $I_t \leftarrow$  Array()
      end if
       $I_t.append(n)$ 
    end for
  end for
  return  $I$ 
end procedure

```

- ▷ D is a set of text documents
 - ▷ Inverted list storage
 - ▷ Document numbering
- ▷ Parse document into tokens

Python solution

- For each document in the “inputpath” folder, it firstly finds the “docid” in tag <newsitem>;
- For each index term (excluding punctuations, and tags <p> and </p>) in <text>, it insert the term into the Index or accumulates its frequency.

```
<?xml version="1.0" encoding="iso-8859-1" ?>
<newsitem itemid="809481" id="root" date="1997-08-19" xml:lang="en">
<title>CANADA: Sherritt to buy Dynatec, spin off unit.</title>
<headline>Sherritt to buy Dynatec, spin off unit.</headline>
<dateline>TORONTO 1997-08-19</dateline>
<text>
<p>Sherritt International Corp said on Tuesday that its wholly-owned
metallurgical technologies business plans to acquire all outstanding shares of
Dynatec International Ltd, a private mining and drilling services company, and
spin off the merged company.</p>
<p>The new company to be formed by the merger of Sherritt International
Consultants Inc and Dynatec will be called Dynatec Corp.</p>
<p>Sherritt said it is filing with Canadian securities regulators a preliminary
prospectus for a divided-in-kind to its shareholders of all of its interest in
Dynatec, after which it will no longer hold any Dynatec securities.</p>
<p>Dynatec Corp will be a publicly traded Canadian company, with Dynatec Intl's
management retaining a significant equity interest in the company.</p>
<p>W. Robert Dengler will be president and chief executive officer of Dynatec,
Sherritt said.</p>
<p>Sherritt's Cuba interests have caused it problems because of U.S. trade laws
that penalize such activities.</p>
<p>((Reuters Toronto Bureau (416) 941-8100))</p>
</text>
<copyright>(c) Reuters Limited 1997</copyright>
</newsitem>
```

CRICOS No.00213J

```
def index_docs(inputpath, stop_words):
    Index = {} # initialize the index
    os.chdir(inputpath)
    for file_ in glob.glob("*.xml"):
        start_end = False
        for line in open(file_):
            line = line.strip()
            if (start_end == False):
                if line.startswith("<newsitem "):
                    for part in line.split():
                        if part.startswith("itemid="):
                            docid = part.split("=")[1].split("\\"")[1]
                            break
                if line.startswith("<text>"):
                    start_end = True
            elif line.startswith("</text>"):
                break
            else:
                line = line.replace("<p>", "").replace("</p>", "")
                line = line.translate(str.maketrans(string.punctuation,
                                                    ' '*len(string.punctuation)))
                for term in line.split():
                    term = stem(term.lower())
                    if len(term) > 2 and term not in stop_words:
                        try:
                            try:
                                Index[term][docid] += 1
                            except KeyError:
                                Index[term][docid] = 1
                        except KeyError:
                            Index[term] = {docid:1}

    return Index
```

16

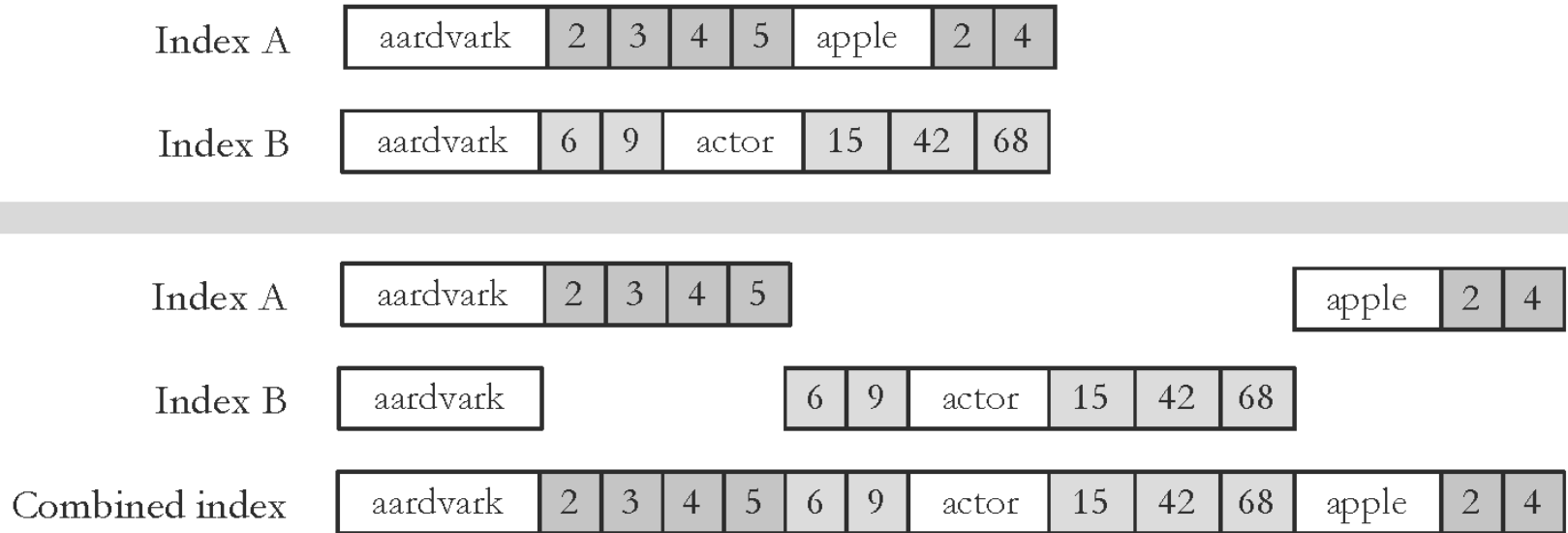
Output of the Index

```
{ 'lehto': {'741299': 2}, 'finland': {'741299': 1}, 'steve': {'741299': 1},  
  ...,  
  'roqu': {'6146': 1}, 'fernandez': {'6146': 1}, 'monetari': {'6146': 1},  
  'deleg': {'6146': 1}, 'fiscal': {'6146': 1}, 'deficit': {'6146': 1}, 'axel': {'6146': 1},  
  'bugg': {'6146': 1}, 'bueno': {'6146': 1}, 'air': {'6146': 1}, 'newsroom': {'6146': 1},  
  '541': {'6146': 1}, '318': {'6146': 1}, '0668': {'6146': 1}  
}
```

Merging

- It is classic way to solve the memory problem if the inverted list is very big (or in distributed index framework).
- Merging addresses limited memory problem to
 - Build the inverted list structure until memory runs out
 - Then write the partial index to disk, and start making a new one again.
 - At the end of this process, the disk is filled with many partial indexes, which are then merged to a single result.
- Partial lists must be designed so they can be merged in a small memory
 - e.g., storing in alphabetical order

Merging cont.



Read two words $w1$ and $w2$ from both Index A and Index B respectively, and continue this process until there are no remaining words in either index.

1. If $w1 = w2$, merging $w1$ and $w2$ and then append it to Index;
2. If $w1 < w2$, append $w1$ to Index; then read a new word from Index A as $w1$ then repeat the process;
3. If $w2 < w1$, append $w2$ to Index; then read a new word from Index B as $w2$ then repeat the process;

Result Merging

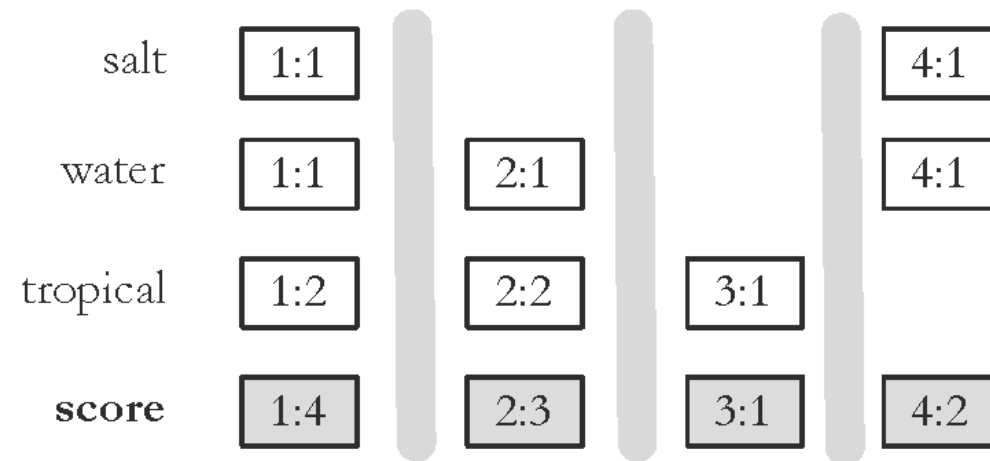
- This merging strategy also shows a possible parallel indexing strategy.
 - If many machines build their own partial indexes; a single machine can combine all those indexes together into the final index.
- Index merging is also a reasonable update strategy when index updates come in large batches, perhaps many thousands of documents at a time.
 - For small updates this is very inefficient
 - instead, create separate index for new documents, merge *results* from both searches.
 - could be in-memory, fast to update and search.

3. Query Processing

- Once an index is built, we need to know how to process it to get query results.
- Clever algorithm can boost query processing speed [1]
 - Document-at-a-time
 - Calculates complete scores for documents by processing all term lists, one document at a time
 - Term-at-a-time
 - Accumulates scores for documents by processing term lists one at a time
- Both approaches have optimization techniques that significantly reduce time required to generate scores

Document-at-a-time

- Document-at-a-time retrieval is a simple way to perform retrieval with an inverted file.
- The following figure shows document-at-a-time retrieval for the query “salt water tropical”.
- The inverted lists are shown horizontally, where the numbers (x:y) represent a document number (x) and a word count (y).
- In the first step, all the counts for the first document are added to produce the score for that document. Once the scoring for the first document has completed, the second document is scored, then the third, and then the fourth.



Document-At-A-Time

procedure DOCUMENTATATIMERETRIEVAL(Q, I, f, g, k)

$L \leftarrow \text{Array}()$

$R \leftarrow \text{PriorityQueue}(k)$

for all terms w_i in Q **do**

$l_i \leftarrow \text{InvertedList}(w_i, I)$

$L.\text{add}(l_i)$

end for

for all documents $d \in I$ **do**

$s_d \leftarrow 0$

for all inverted lists l_i in L **do**

if $l_i.\text{getCurrentDocument}() = d$ **then**

$s_d \leftarrow s_d + g_i(Q)f_i(l_i)$

▷ Update the document score

end if

$l_i.\text{movePastDocument}(d)$

end for

$R.\text{add}(s_d, d)$

end for

return the top k results from R

end procedure

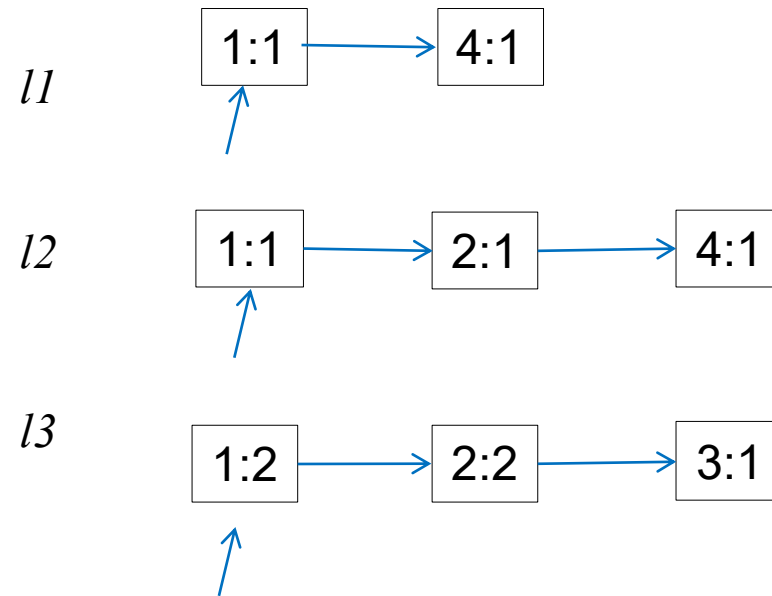
\\Select inverted lists based on the query

$f_i(d)$

Example 1:

$Q = \{\text{salt} = l1, \text{water} = l2, \text{tropical} = l3\}$

Documents = {d1, d2, d3, d4}



For d1:

$$s_d = 0$$

$$s_d = s_d + 1 = 1$$

$$s_d = s_d + 1 = 2$$

$$s_d = s_d + 2 = 4$$

For d2:

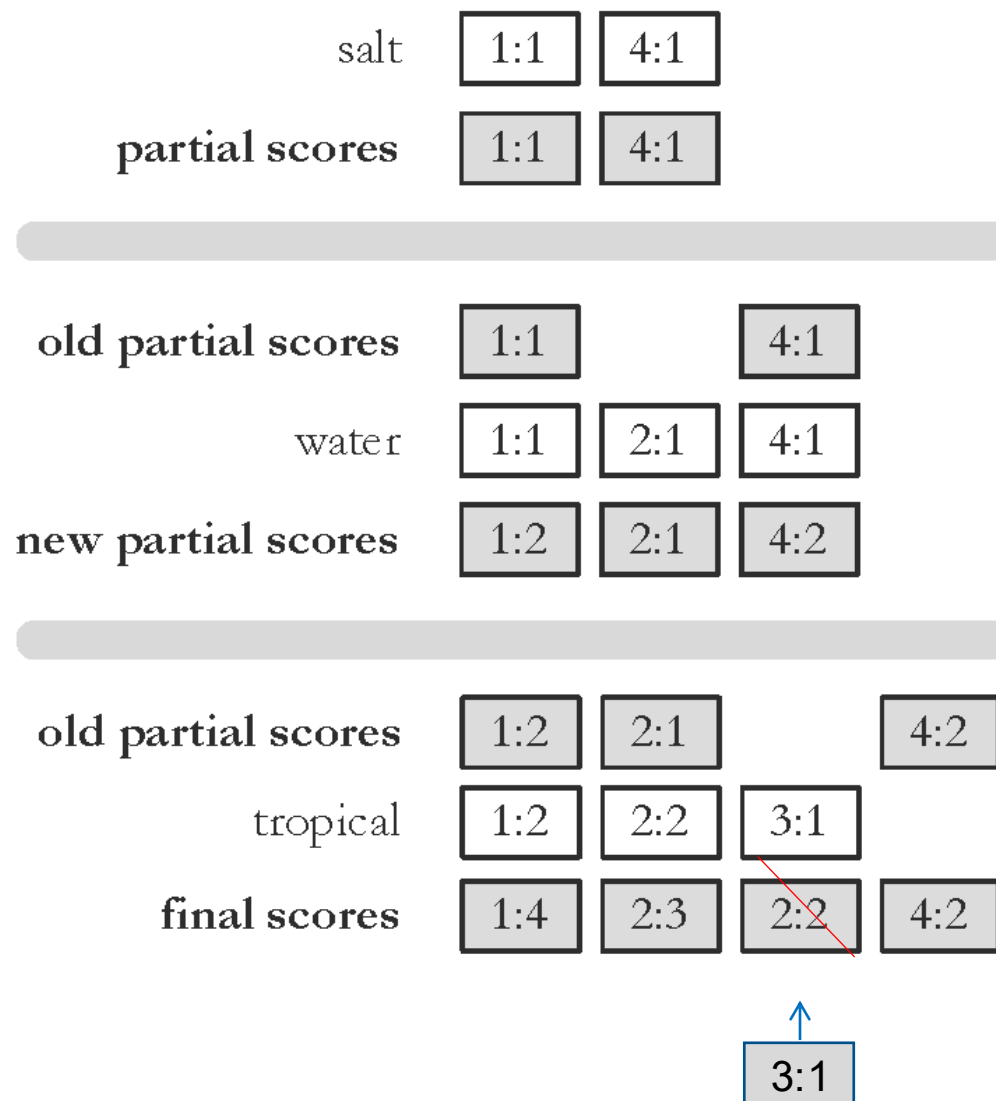
$$s_d = 0$$

$$s_d = s_d + 1 = 1$$

$$s_d = s_d + 2 = 3$$

Term-at-a-time

- We use the same query, scoring function, and inverted list data as in the document-at-a-time example.
- In the first step, the inverted list for “salt” is recoded, and partial scores are stored in accumulators.
- These scores are called partial scores because they are only a part of the final document score.
- In the second step, partial scores from the accumulators are combined with data from the inverted list for “water” to produce a new set of partial scores.
- After the data from the list for “tropical” is added in the third step, the scoring process is complete.

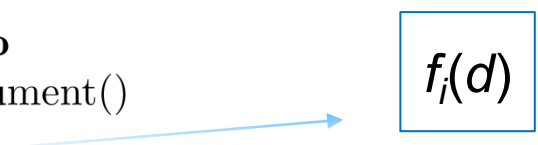


Term-At-A-Time

```

procedure TERMATATIMERETRIEVAL( $Q, I, f, g, k$ )
   $A \leftarrow \text{HashTable}()$ 
   $L \leftarrow \text{Array}()$ 
   $R \leftarrow \text{PriorityQueue}(k)$ 
  for all terms  $w_i$  in  $Q$  do
     $l_i \leftarrow \text{InvertedList}(w_i, I)$ 
     $L.\text{add}(l_i)$ 
  end for
  for all lists  $l_i \in L$  do
    while  $l_i$  is not finished do
       $d \leftarrow l_i.\text{getCurrentDocument}()$ 
       $A_d \leftarrow A_d + g_i(Q)f(l_i)$ 
       $l_i.\text{moveToNextDocument}()$ 
    end while
  end for
  for all accumulators  $A_d$  in  $A$  do
     $s_d \leftarrow A_d$ 
     $R.\text{add}(s_d, d)$ 
  end for
  return the top  $k$  results from  $R$ 
end procedure

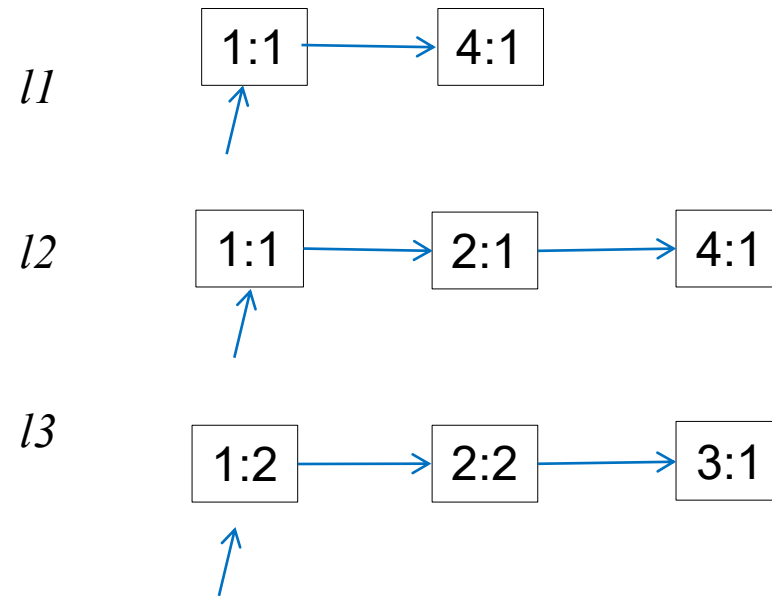
```



Example 2:

$Q = \{\text{salt} = l1, \text{water} = l2, \text{tropical} = l3\}$

Documents = {d1, d2, d3, d4}



HashTable A

$$A1 = 0 + 1 = 1$$

$$A4 = 0 + 1 = 1$$

$$A1 = A1 + 1 = 2$$

$$A2 = 0 + 1 = 1$$

$$A4 = A4 + 1 = 2$$

$$A1 = A1 + 2 = 4$$

$$A2 = A2 + 2 = 3$$

$$A3 = 0 + 1 = 1$$

4. Information Extraction

- Information extraction is a language technology that focuses on extracting structure from text, e.g., Identifying noun phrases, titles, or even bolded text.
 - Automatically extract structure from text
 - annotate document using tags to identify extracted structure
 - *Named entity recognition*
 - identify words that refer to something of interest in a particular application
 - e.g., people, companies, locations, dates, product names, prices, etc.
- Most of the recent research in information extraction has been concerned with features that have specific semantic content, such as named entities, relationships, and events.

Named Entity Recognition

- Named Entity Recognition is the process of recognizing entities and tagging them in text is sometimes called semantic annotation = labels.
- The following Example shows semantic **annotation** of text using XML tags:

Fred Smith, who lives at 10 Water Street, Springfield, MA, is a long-time collector of **tropical fish**.

```
<p ><PersonName><GivenName>Fred</GivenName> <Sn>Smith</Sn>
</PersonName>, who lives at <address><Street >10 Water Street</Street>,
<City>Springfield</City>, <State>MA</State></address>, is a long-time
collector of <b>tropical fish.</b></p>
```

- Information extraction also includes document structure and more complex features such as *relationships* and *events*.

Traditional Approaches

• **Rule-based**

- Uses *lexicons* (lists of words and phrases) that categorize names
 - e.g., locations, peoples' names, organizations, etc.
- Rules also used to verify or find new entity names
 - e.g., "<number> <word> street" for addresses
 - "<street address>, <city>" or "in <city>" to verify city names
 - "<street address>, <city>, <state>" to find new cities
 - "<title> <name>" to find new names
- Rules either developed manually by trial or using machine learning techniques.

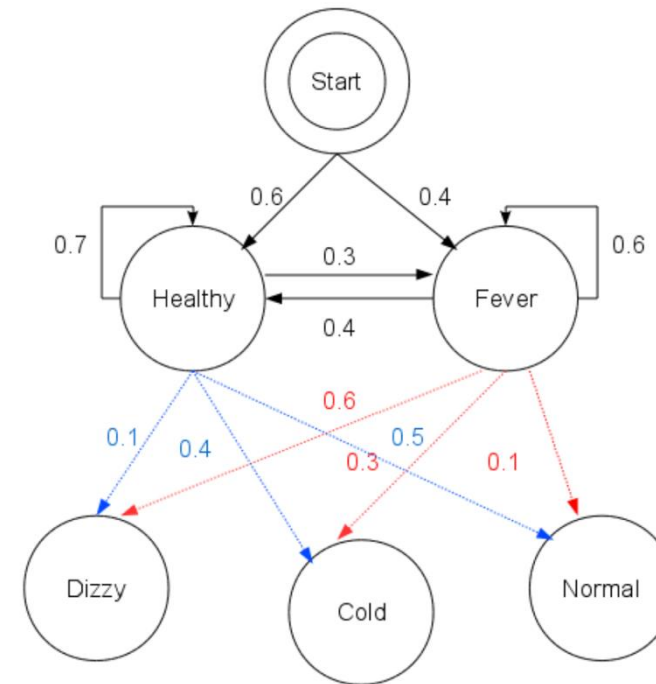
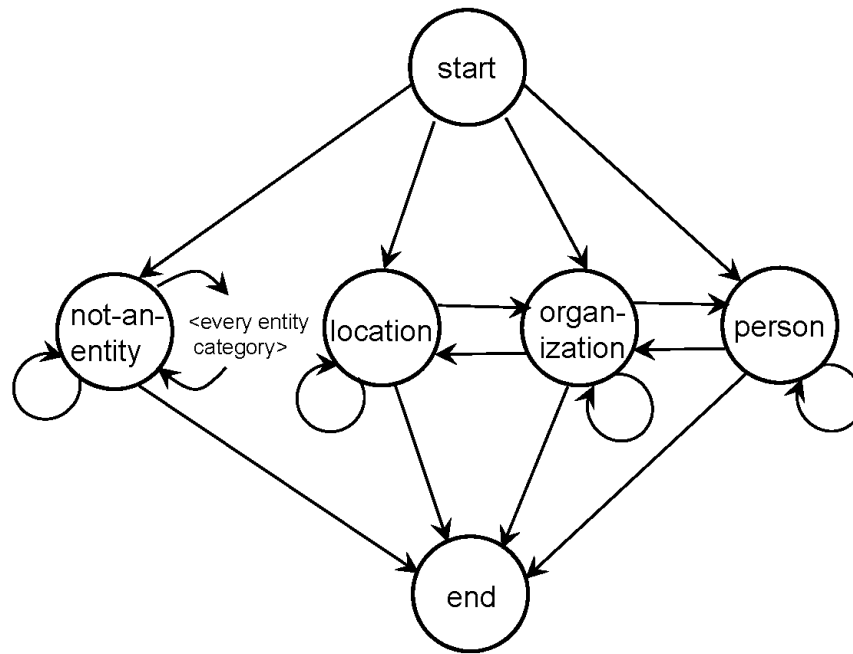
• **Statistical Approach**

- Uses a probabilistic model of the words in and around an entity
- Probabilities estimated using *training data* (manually annotated text)
- Hidden Markov Model (HMM) is one approach
 - Resolve ambiguity in a word using *context (the words that surround it)*
 - e.g., "marathon" is a location or a sporting event, "boston marathon" is a specific sporting event
- Model context using a *generative* model of the sequence of words
 - *Markov property*: the next word in a sequence depends only on a small number of the previous words

5. Hidden Markov Model (HMM)

- *Markov Model (MM)* describes a process as a collection of states with transitions between them
 - each transition has a probability associated with it
 - next state depends only on current state and transition probabilities
- *Hidden Markov Model (HMM)*
 - each state has a set of **possible outputs**
 - outputs have probabilities

Examples of MM and HMM



- The left is a MM to describe the possible states (categories) of a sentence. It can be extended to an HMM if each state is associated with a probability distribution over words (the output).
- The right one is an HMM which assumes a patient has two states: "Healthy" and "Fever". A doctor cannot see the (hidden) states directly; but the patient can tell the doctor that she/he is "normal", "cold", or "dizzy" (the observations).

HMM for Extraction

- Could generate sentences with this model
- To recognize named entities, find sequence of “labels” that give highest probability for the sentence
 - only the outputs (words) are visible or observed
 - states are “hidden”
 - e.g., <start><name><not-an-entity><location><not-an-entity><end>
- *Viterbi* algorithm used for recognition

Inputs and output of HMM

- Inputs
 - The observation space O , e.g., O is a set of words.
 - The state space S , e.g., S includes entities, such as, “location”, “person”, “organization” and “not-an-entity”; and the initial probabilities p_i of each state s_i or entity.
 - Transition matrix A , where A_{ij} is the transition probability of transiting from state s_i to state s_j .
 - Emission matrix B , where B_{ij} is the probability of observing o_j from state s_i
 - a sequence of observation Y (a sub-set of O in order)
- Output
 - The most likely hidden state sequence X

Inputs and output of HMM cont.

35

Initial probabilities

$p_1 \quad p_2 \quad \dots \quad p_K$

States

$s_1 \quad s_2 \quad \dots \quad s_K$

s_1	A_{11}	A_{12}	$\dots$	A_{1K}
s_2	A_{21}	A_{22}	$\dots$	A_{2K}
$\dots$	$\dots$			$\dots$
s_K	A_{K1}	A_{K2}	$\dots$	A_{KK}

← A

INPUTS: (1) We have K states and each state s_i is assigned an initial probability p_i . The relations between states form a graph (Transition matrix A), where A_{ij} is the transition probability of transiting from state s_i to state s_j (2) We also have N observations o_j in total, and the relations between observation and states form a graph (Emission matrix B), where B_{ij} is the probability of observing o_j from state s_i (3) A sequence y_1 to y_T

$y_1 \quad \dots \quad y_T$

Input Sequence

$o_1 \quad o_2 \quad \dots \quad o_N$

All observations

s_1	B_{11}	B_{12}	$\dots$	B_{1N}
s_2	B_{21}	B_{22}	$\dots$	B_{2N}
$\dots$	$\dots$			$\dots$
s_K	B_{K1}	B_{K2}	$\dots$	B_{KN}

← B

Output Sequence (X) = $x_1, x_2, \dots, x_T$

6. LLM-Based Information Extraction

- Large Language Models (LLMs), such as GPT-4 (OpenAI, 2024), ChatGPT (OpenAI, 2023), and Llama 2 (Touvron et al., 2023), have significantly advanced the field of NLP, and led to a surge of interest in generative IE methods [2].
- LLMs enable a new paradigm for information extraction by leveraging deep contextual and semantic understanding of text, rather than relying on manually designed rules or fixed feature representations.
- It supports **Prompt-Based Extraction**
 - Information extraction can be formulated as a **natural language prompting** problem:
 - **Input:** raw text + extraction instructions
 - **Output:** structured results (JSON, tables, key–value pairs)

Example of Prompt-Based Extraction

- **An example prompt**

You are an information extraction system. Extract the following entities from the text.

Return the result strictly in valid JSON using this schema:

```
{  
  "Person": [string],  
  "Organization": [string],  
  "Location": [string],  
  "Date": [string]  
}
```

Text: "Prof Y. Li (QUT), who also served as a Program Co-Chair, chaired Session 3, Research Track: Deep Learning Fusion and Vision, at AusDM 2025, held in Brisbane on 27 November 2025."

Example cont:

- ChatGPT output:

json

 Copy code

```
{  
  "Person": ["Prof Y. Li"],  
  "Organization": ["QUT", "AusDM 2025"],  
  "Location": ["Brisbane"],  
  "Date": ["27 November 2025"]  
}
```

Summary

- Accurate recognition requires about 1M words of training data (1,500 news stories)
 - may be more expensive than developing rules for some applications
- Both rule-based and statistical approaches can achieve about 90% effectiveness for categories such as names, locations, organizations
 - others, such as product name, can be much worse.
- LLM-Based methods treats information extraction as a language understanding and generation problem.
 - They use prompting instead of architecture redesign
 - LLM-based IE is not just a better extractor. It is a paradigm shift.

Reference

- [1] Chapter 5 in book - W. Bruce Croft, *Search Engines - Information retrieval in Practice*; Pearson, 2010.
- [2] Liu Pai, and et al., 2024. [A Survey on Open Information Extraction from Rule-based Model to Large Language Model](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 9586–9608, [10.18653/v1/2024.findings-emnlp.560](#)